

iResolve Module Development Framework — Implementation Spec

0. Goal

Enable other teams to build new iResolve modules (module 6, 7, 8, ...) **without ever seeing or touching iResolve's core source code**, and without the core owner ever having to modify core code to onboard a new module. A team receives an SDK, builds a module folder against it, hands it back, and it merges into iResolve with near-zero manual wiring.

Non-negotiables:

- Core code is **never edited** to add a module. Adding a module = adding a folder under `modules/<id>/` + one registry row.
- The SDK is **self-sufficient**: auth (Windows Auth/LDAP/ADFS SSO), role/permission checks, DB access, Celery, logging, and UI primitives are all available through the SDK — a module author never needs core internals.
- The SDK is **regenerated fresh from core on every production build**, so it can never silently drift from what core actually does.
- Modules are **isolated by contract** (namespacing, DB schema, permission keys) so a bad module can't corrupt or read another module's data, and can't be wired in without passing validation.

1. Repository layout

```
iresolve/
  core/                                # existing core app — NEVER shared, NEVER edited t
  modules/
    <module_id>/                       # e.g. 06_qa, 07_rca_capa, 08_letters
      manifest.json
      backend/
        router.py                     # exposes `router: APIRouter`
        tasks.py                      # optional Celery tasks
        models.py                     # SQLAlchemy models, own schema only
        migrations/                  # Alembic revisions scoped to this module's sch
```

```

    frontend/
        index.tsx                # lazy-loaded entry, default export
        manifest.ts              # nav entry, route, required permission key
    sdk/
        python/iresolve_sdk/     # generated package, versioned
        ts/@iresolve/sdk/        # generated package, versioned
    build/
        build_sdk.ps1
        build_production.ps1
        validate_module.ps1

```

2. SDK contents (must be complete — this is the whole point)

2.1 Python SDK (`iresolve_sdk`)

- `iresolve_sdk.auth`
 - `get_current_user()` — FastAPI dependency, resolves the Windows Auth / ADFS SSO identity into a typed `User` object (id, username, ldap_groups, display_name).
 - `get_ldap_groups(user)` — returns the caller's AD/LDAP group membership, already resolved.
 - `require_permission(permission_key: str)` — FastAPI dependency factory; 403s if the current user lacks `<module_id>.<permission_key>` in the existing PostgreSQL-backed permission table.
 - `require_ldap_group(group: str)` — dependency factory for group-gated endpoints, for modules that need raw AD group checks instead of/alongside the permission table.
- `iresolve_sdk.db`
 - `get_db()` — yields a SQLAlchemy session **scoped to the module's own Postgres schema** (connection-level `search_path` set to `<module_id>`, plus GRANT-enforced at the DB level — not just app-level trust).
 - `get_shared_readonly_db()` — read-only session against explicitly whitelisted shared/common tables (e.g. account lookups), if a module needs cross-module reference data.
- `iresolve_sdk.tasks`
 - `celery_task(name: str, **kwargs)` — decorator wrapping the existing

Celery/Redis app; enforces task names are prefixed `<module_id>.*`.

- `iresolve_sdk.module`
 - `BaseModule (ABC)` — every module's `backend/__init__.py` implements this:
`router, manifest, on_register(app)` hook.
- `iresolve_sdk.logging`
 - `get_logger(module_id)` — standard structured logger wired into core's existing logging/monitoring.
 - `WORKING_PHRASES` — access to the shared branded phrase bank for
`<WorkingIndicator />` -equivalent backend job status messages.
- `iresolve_sdk.reporting`
 - Wrapper around the existing async docx/PDF reporting pipeline (Celery-backed), so modules that generate reports (like the SAST/DAST or RCA/CAPA modules) don't reimplement it.
- `iresolve_sdk.contracts`
 - Pydantic models: `ModuleManifest, PermissionDef, NavEntry` — the typed contract every `manifest.json` must satisfy.

2.2 TypeScript SDK (@iresolve/sdk)

- `useAuth()` — current user, LDAP groups, roles.
- `usePermission(key: string)` — boolean gate matching the backend
`require_permission` key.
- `apiClient` — pre-configured fetch/axios wrapper (base URL, auth headers, error handling) scoped to `/api/modules/<module_id>/*`.
- `<ModuleShell>` — layout wrapper matching core's chrome (nav, header) so module UIs are visually consistent without importing core components directly.
- Shared UI kit (buttons, tables, forms, modals) matching current design tokens.
- `WorkingIndicator` component, pre-wired to the shared phrase bank.
- Type-only `ModuleManifest` interface mirroring the Python contract.

2.3 What the SDK deliberately does NOT expose

- No other module's code or data.

- No core routing/wiring code — modules never edit core.
 - No raw core DB credentials — only the scoped session.
 - No direct ADFS/LDAP server config — only resolved identity/group data.
-

3. `manifest.json` contract (per module)

```
{
  "module_id": "07_rca_capa",
  "name": "RCA / CAPA",
  "version": "1.0.0",
  "sdk_version": "2.3.0",
  "owner_team": "string",
  "nav": { "label": "string", "icon": "string", "path": "/modules/07_rca_capa" },
  "permissions": [
    { "key": "07_rca_capa.view", "label": "string" },
    { "key": "07_rca_capa.approve", "label": "string" }
  ],
  "ldap_groups": ["optional AD group names, if used directly"],
  "db_schema": "07_rca_capa",
  "celery_task_prefix": "07_rca_capa",
  "migrations_path": "backend/migrations"
}
```

The loader and validator both parse against this schema — this is the single source of truth for how a module plugs in.

4. Auto-discovery / zero-wiring loader (core-side, built once)

On startup, core:

1. Scans `modules/*/manifest.json`.
2. Validates each against `ModuleManifest` contract; refuses to mount on schema mismatch or `sdk_version` incompatibility (see §6).
3. Imports `backend/__init__.py`, calls `BaseModule.on_register(app)` to mount `router` under `/api/modules/<module_id>`.
4. Registers permissions from `manifest.permissions` into the existing PostgreSQL

permission table (idempotent upsert).

5. Registers Celery tasks found in `tasks.py`.
6. Frontend: shell reads the same manifests (via a generated `modules.json` index) to build nav + lazy-load routes with `React.lazy()`.

This loader is the **only** core code that ever needs to exist for module support — once built, adding module 9 requires zero core changes.

5. DB isolation

- Each module gets its own Postgres **schema** named after `db_schema`.
 - `GRANT` is scoped so a module's DB role can only touch its own schema, plus any explicitly whitelisted shared read-only views.
 - Each module ships its own Alembic branch (`migrations_path`), applied independently at merge time — never touches core's or another module's migration history.
-

6. SDK versioning & compatibility

- SDK is semver'd (`sdk_version`). Every module manifest declares the SDK version it was built against.
 - The loader compares `manifest.sdk_version` to core's currently-running SDK version and refuses to mount on a breaking mismatch (major version bump).
 - **The SDK is regenerated from core on every production build** (see `build_sdk.ps1` below), so it's always an accurate reflection of core's current auth/db/permission internals — teams are never handed a stale or hand-maintained SDK.
-

7. Build pipeline (PowerShell)

7.1 `build_sdk.ps1`

Runs standalone, or as a stage inside `build_production.ps1`. Responsibilities:

1. Bump/compute the next SDK semver (based on a changelog file or manual flag).

2. Regenerate `sdk/python/iresolve_sdk` from core's actual `auth/db/permission/celery/reporting` modules (thin wrapper layer, not a copy of core internals) — this step is the one piece of core-aware codegen and should assert it produces **no direct exposure of core business logic**.
3. Regenerate `sdk/ts/@iresolve/sdk` similarly from core's TS types/hooks/UI kit.
4. Package Python SDK as a wheel, TS SDK as an npm tarball.
5. Stamp both with the new `sdk_version`.
6. Publish to your internal package location (private feed, or a drop folder if no package registry is available given the no-third-party-install constraint).

7.2 `validate_module.ps1`

Run against a submitted module folder before merge:

1. Parse and validate `manifest.json` against the contract schema.
2. Confirm `sdk_version` compatibility.
3. Static-check: no imports outside `iresolve_sdk` / `@iresolve/sdk` / `stdlib` / approved third-party libs.
4. Confirm all route prefixes, permission keys, Celery task names, and DB schema name are correctly prefixed with `module_id`.
5. Dry-run the Alembic migration against a scratch schema.
6. Emit a pass/fail report.
7. **Update-chain check** (for a module that has been merged before, not a first-time onboarding): confirm the submitted migration folder contains only the *new* revision(s) since the module's last-merged head — i.e. the chain applies cleanly on top of what's currently deployed for that module's schema, not a regenerated full history. Reject if it tries to reapply or redefine already-deployed revisions.

7.3 `build_production.ps1`

Orchestrates, in order:

1. `build_sdk.ps1` (always regenerates latest SDK first, so validation below uses current-truth).
2. `validate_module.ps1` against every folder in `modules/*`.
3. Apply all module Alembic migrations (each to its own schema).

4. Build frontend (core shell + all module bundles via lazy-loaded chunks).
 5. Package backend as a single `.exe` via PyInstaller (existing deployment pattern).
 6. Restart IIS reverse proxy / redeploy service.
 7. Post-deploy smoke test: hit a health endpoint per module.
-

8. Handoff-to-team workflow

1. Team receives: SDK (Python wheel + npm package, latest version) + this spec + a module template folder (empty `manifest.json`, `backend/__init__.py` stub implementing `BaseModule`, `frontend/index.tsx` stub) + a mock harness (minimal FastAPI/React shell simulating auth/db/permissions) for local dev.
 2. Team builds entirely against the SDK, using the mock harness to test locally — no core source ever touches their machine.
 3. Team hands back the completed `modules/<id>/` folder.
 4. Core owner runs `validate_module.ps1` → fixes/returns feedback if it fails.
 5. On pass, folder is copied/subtree-added into the real `modules/` tree, registry row added.
 6. `build_production.ps1` run end-to-end (regenerates SDK, validates, migrates, builds, deploys).
-

8b. Module update workflow (after initial merge)

Same pipeline as first-time onboarding, applied to an already-merged module:

1. Team edits their existing `modules/<id>/` folder — code, frontend, new Alembic revision(s), and bumps `manifest.json`'s `version` (not `sdk_version`, unless they also rebuilt against a newer SDK).
2. Team hands back the updated folder (not a fresh one — same `module_id`, same schema).
3. Core owner runs `validate_module.ps1` — same checks as onboarding, plus the update-chain check (§6.7): new migrations must chain onto the currently-deployed head, not redefine history.

4. If the module was built against an SDK version that's since had a breaking (major) bump, validation fails here with a clear incompatibility message — team rebuilds against latest SDK first.
5. On pass, folder replaces the existing one in `modules/`, registry row's version bumped.
6. `build_production.ps1` run as usual: regenerates SDK, validates all modules, applies only the new migrations (each module's schema is isolated, so this never touches other modules), builds, deploys, smoke-tests.

No new tooling required — updates are just re-runs of the same onboarding pipeline against a module that already exists.

9. Open decisions before implementation starts

- Package distribution: private feed (Azure Artifacts/Nexus) vs. drop-folder wheel/tarball, given the no-third-party-install constraint on the server.
- Module ID convention: numeric prefix (`06_qa`) vs. semantic slug only — affects nav ordering.
- Whether module authors get a staging environment against real core, or only the mock harness, before final handoff.
- Deprecation policy when SDK major-version bumps break older unmerged modules.